

1. Reliable Scalable & Maintainable Applications

Start

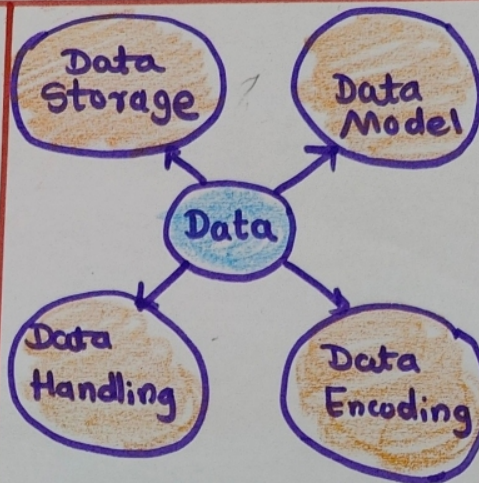
Thinking About Data

Simple Data System

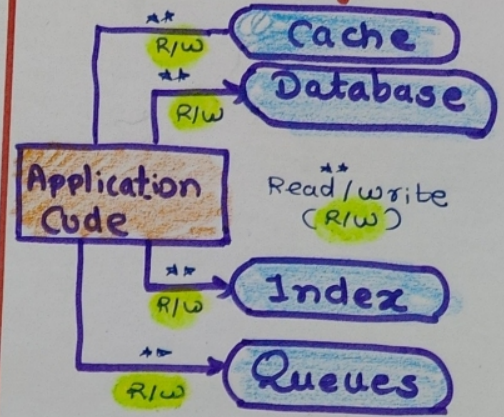
Reliability

Scalability

Maintainability



Simple Data System

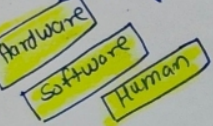


Reliability***

Fault vs Failure

Bug / Small functionality issue

Entire System Collapse



Type-fault	Example(s)	How to handle	Summary
Hardware fault	Disk, Ram, Power, N/w, Fan outages	Test / Prevent / Handle / anticipate / Tolerate / add redundancy	NO 100% Protection
Software faults	1> slw bug 2> Long running process 3> slow down stream service	1> Make right assumptions 2> Test / anticipate 3> Monitor / Analyze 4> Restart 5> fix	Follow best practice throughout SDLC
Human Errors	1> operator error 2> Bad CR 3> unintended action	1> Training 2> Best Practices 3> 2 PR / Reviews	1> Avoidable & improvable thr / best practices

Scalability

Ability to Cope up with Increased Load

Access Patterns

Describe performance

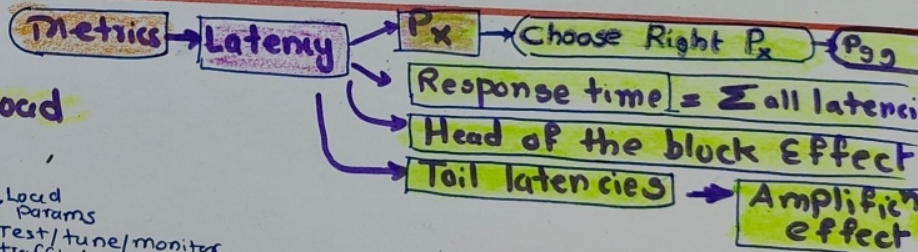
Throughput
Response time
P_x latency

Describe Load

Load Params
Test / tune / monitor traffic / H/w limit / API limit / View count etc

Cope up with Load

Scale (Horizontal vs Vertical)
Precompute
Batch vs Realtime decision
No magic scaling source



Normal case

Celebrity case

Realtime way

Batch way

On the fly compute

Precompute

Maintainability

Simplicity

Maintainability

Operability

Evolvability

Operability

On-call Routine
Anticipate Problems
Fix those
Documentation & Best Practices

Simplicity

Simple, Modular, loosely coupled
Abstraction, Best coding
ACBDA style Layered architecture

Evolvability

Requirement deep dive / Early fix
Agile / Iterative methodology / TDD
Periodic feedback / Adaptability
Refactoring as needed

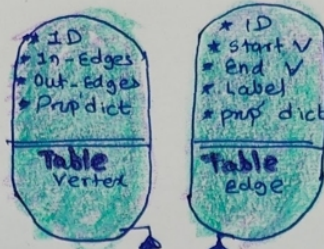
2. Data Models and Query Languages

Summary

Types of data Model

- Relational model
- Document model
- Graph Model
- Hybrid Model

Example - MapReduce
Partly Declarative & Imperative
Partly SQL & NoSQL



Property Graph
Triple store

Abstraction / Parallelization
Concise / Loose Coupled

Ordering Guarantee

Relationship

Many to One

One to Many

Many to Many

Declarative QL

vs

Imperative QL

Relational vs Document Model

Parameter	Relational Model	Document Model
Relationship	Many to one (multiple tables, foreign keys)	One to Many
Query Language	Declarative (e.g. SQL)	Imperative (e.g. javascript)
Data Locality	Less (Normalized data with LESS redundancy)	High (Almost 100%)
Normalization	High (Foreign key usage across tables)	Low (Almost No)
Schema type (nature)	Complex → On Write (strict Static)	Simple → On Read (Dynamic)
Application type	Complex Evolving App	Simple App
Joins at	DB Engine (Highly abstract Query Processing at DB engine)	Application Code
Abstraction Level	Very High (App code has nothing to do with DB engine)	Very Low
Impedance Mismatch	High (Evolved schema leading to App & DB Schema mismatch)	Low
Optimized For	RDBMS / SQL - (M2one)	NoSQL (One2M)
Data Size	Mid - to - High (Typically less than that of doc)	High - to - Very High
Example	Postgres, Oracle, MS	Cassandra, Hbase, DDB
Processing type	OLTP (transactional)	OLAP (Analytics)

Map-Reduce steps

SELECT Date_Trunc(birthday) FROM Student WHERE family == "pawar" GROUP BY birthday

Declarative (Select)
Declarative (Where)
Imperative (Reduce)

(Map) Imperative
↑
Declarative to Imperative

Graph Models

Property Graph

Triple store Graph

Vertex
Edge

Graph

Tables

Table format

Y E

G

IN out prop
v < ID, [I], [O], [P]

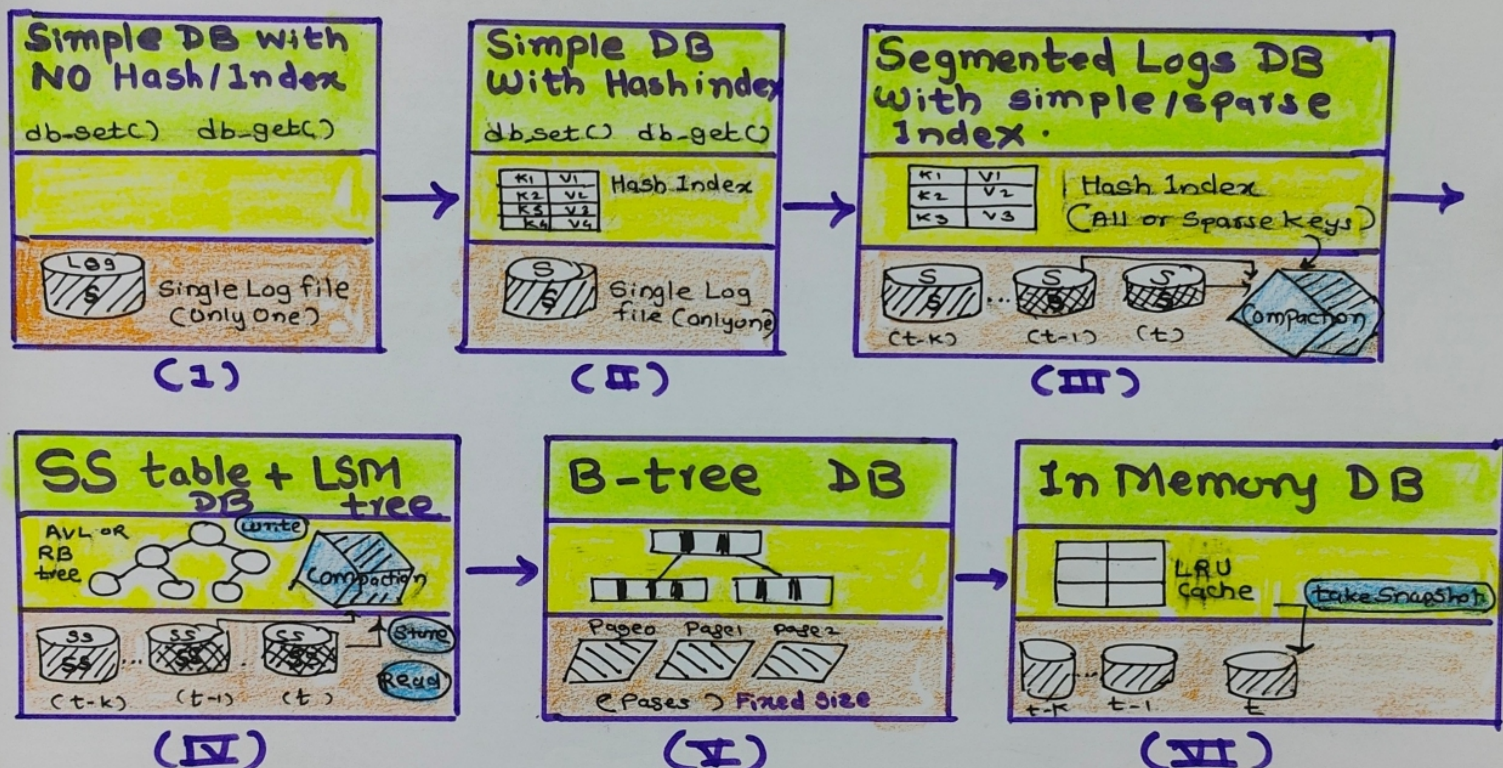
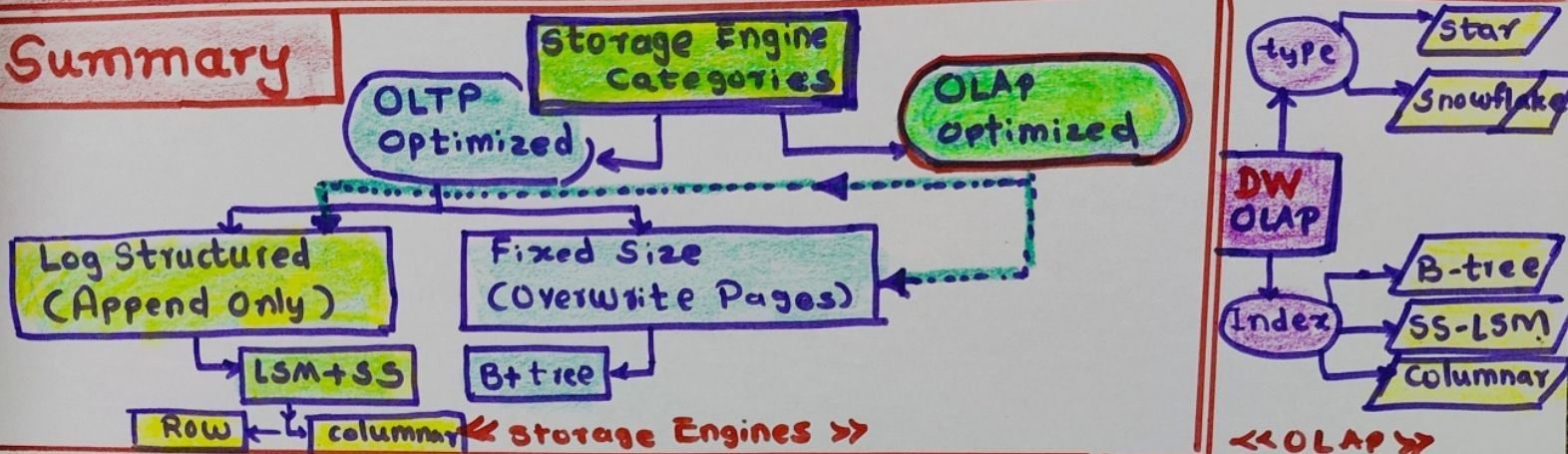
Representation

< subject, Predicate, object

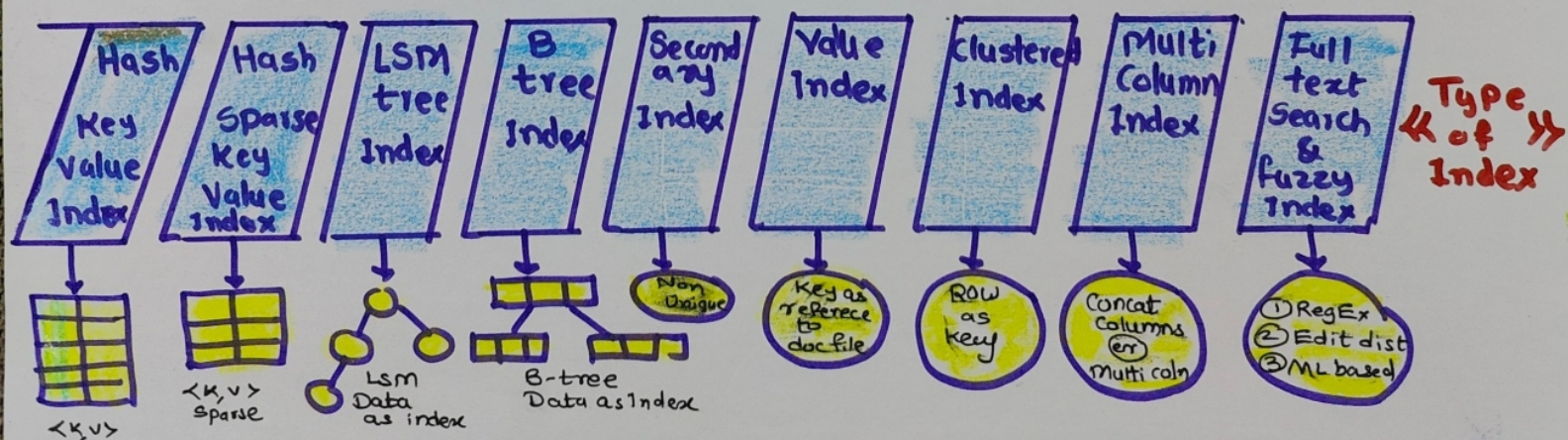
3. Storage and Retrieval

(Part - 1)

Summary



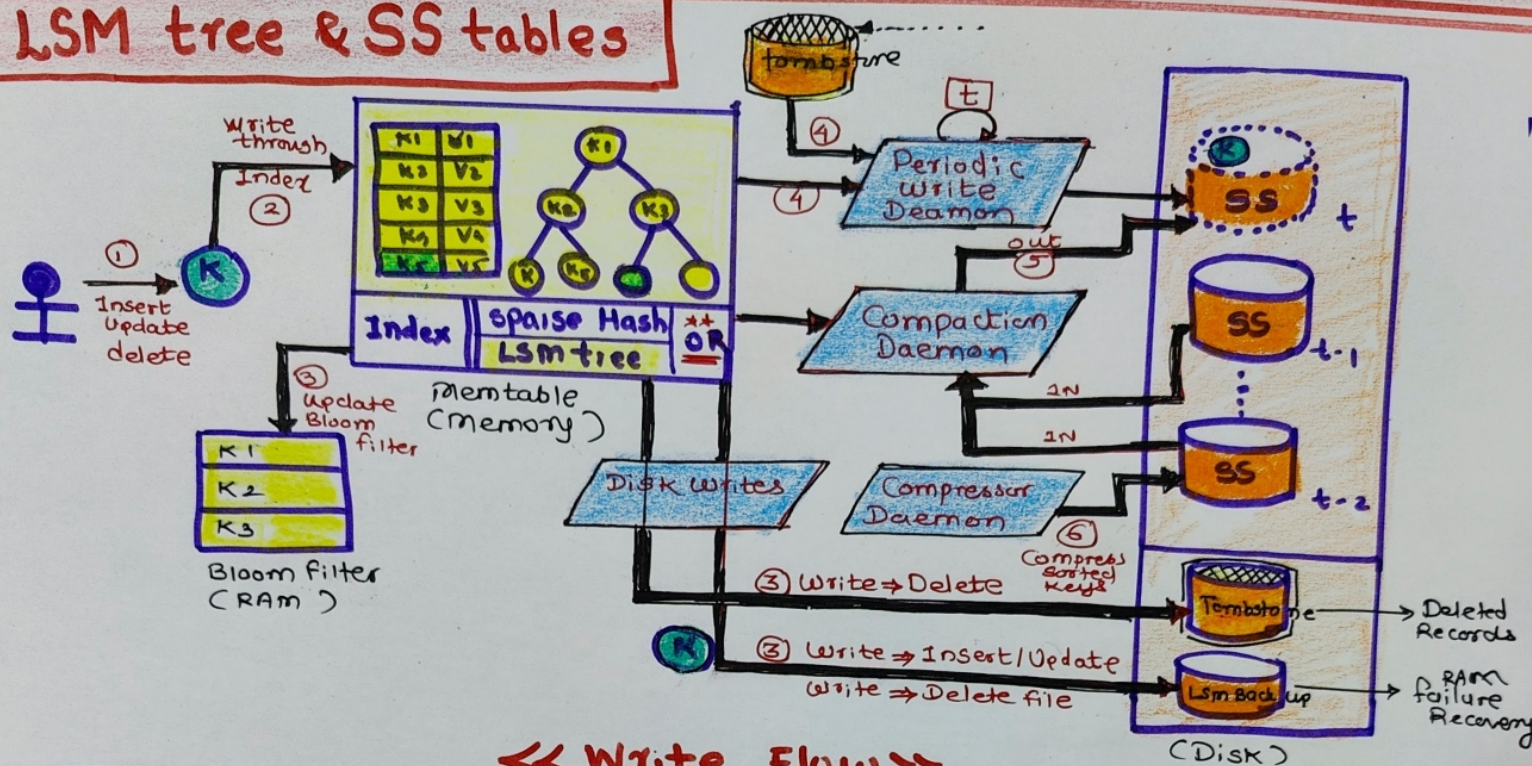
« Evolution of Databases »



3. Storage and Retrieval

(Part-2)

LSM tree & SS tables



Write Flow

- Read-Flow involves → ① LSM → ② Recent Block → ③ older blocks ...
- Bloom filters → Optimizes "costly" disk/SS log search • Stores existing keys
- Compaction types → Size tiered vs Levelled tiered

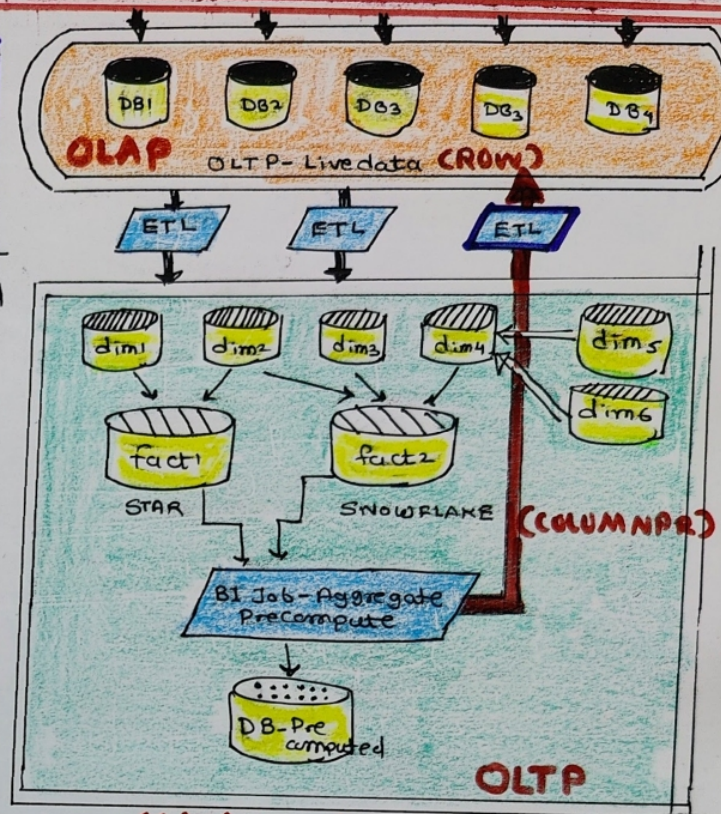
Property	LSM + SS DB	B-tree DB
Efficient for	WRITES (Mem write in realtime)	READ (Multi page write slower)
Concurrency	Easy to achieve	Hard - Require latches
Compaction	(Independent) But Interfere with DB	Does not interfere with DB operations
Disk logs	Append only	Supports overwrites / page update
Disk file size	Variable sized (Logs)	fixed size (Pages) e.g 4K, 8K
Write Complexity	Just insert in to LSM tree & let 'memtable' sync to disk as part of daemon process.	requires page writes, split pages, Update parent page reference, hence Involves 'multiple' disk writes
Major cons	① Compaction Interfere ② slow reads - worst case involves reading all logs ③ Not optimized reads - No fixed size logs in multiple of disk block size ④ Initial compaction consumes more disk bandwidth	① Extremely slow writes compared to LSM-SS DB ② Concurrency requires latches ③ Cascading writes (multi-pages)
Key existence	At multiple logs + tombstone + Bloom filter + in LSM tree	Just inside single Page - that's it.
Transaction dynamics	Weak	Strong
Size driver	Write Daemon run frequency.	"Branching-factor"
Used mainly by	NO SQL DBs	SQL / RDBMS DBs

3. Storage and Retrieval

(Part-3)

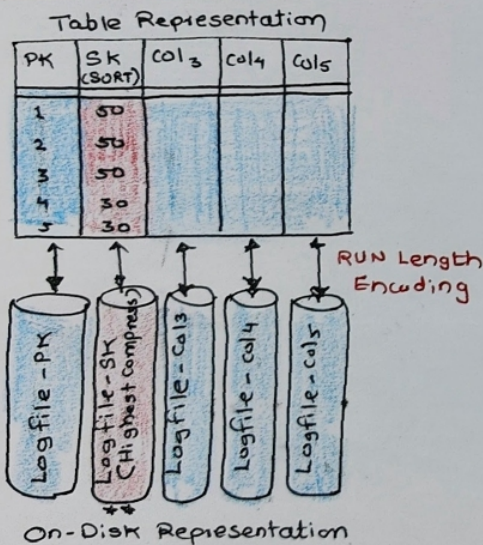
Param	OLTP	OLAP
Read Pattern	① By Key ② Small set of records ③ Multi columns (90/100)	① By range filter ② Huge set of records ③ 4/100 columns
Write Pattern	① Random Writes ② Low latency	① Bulk import-Export ② Event stream based ③ Batch-high latency
User	① End User ② Customer / App	① DW admin ② BI Analyst
Data state	Latest (State)	Historical (Facts)
Row / column	ROW-wise logs	COLUMN wise logs
Dataset size	GB to TB	TB to PB
Sorting	Secondary Index	Sort order

«OLTP vs OLAP»

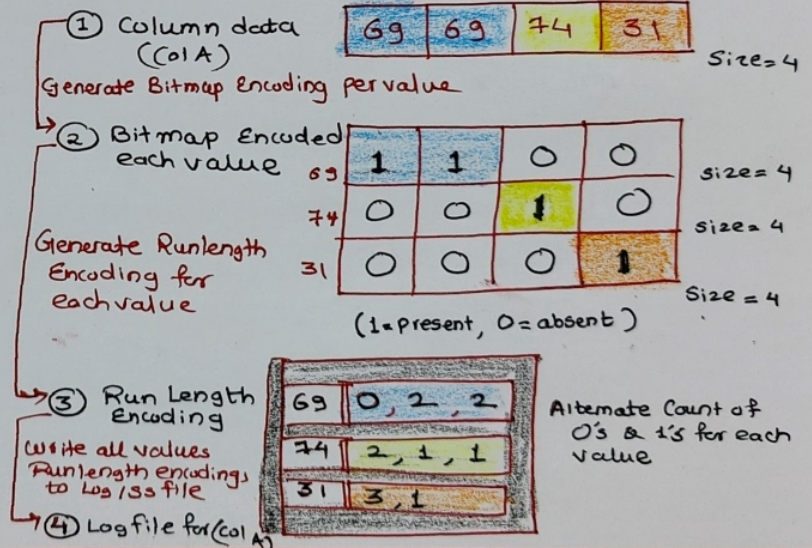


«Interaction»

Column Oriented Storage



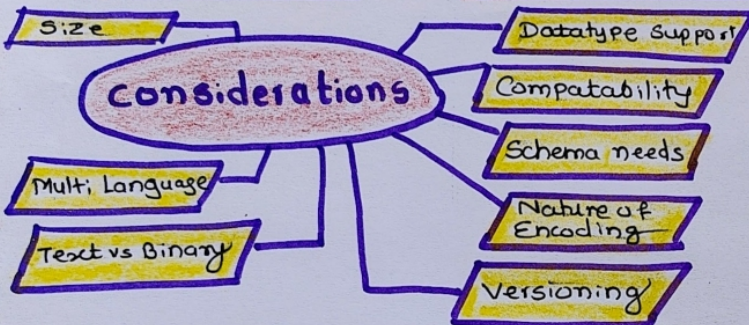
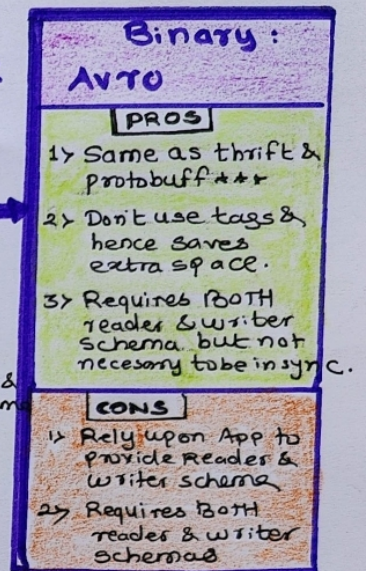
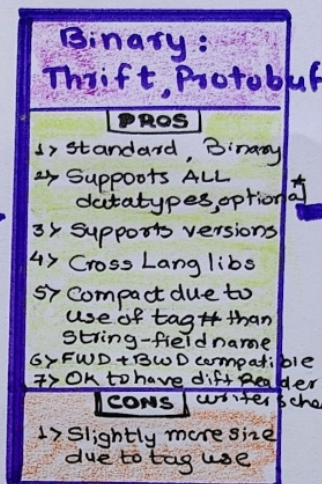
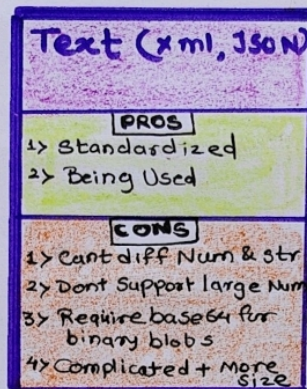
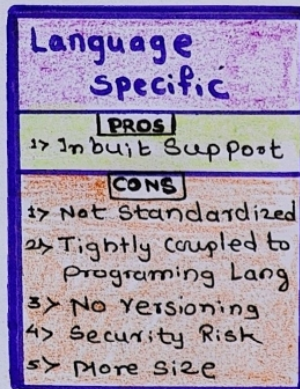
Run Length Encoding



4. Encoding and Decoding

Evolution of Encoding

→ Data Size →



Modes of Dataflows

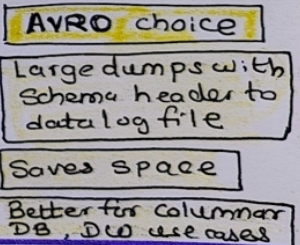
Compatibility

And

Encoding

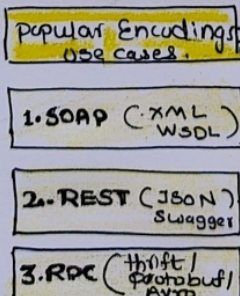
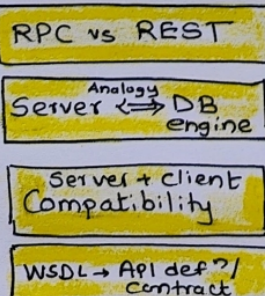
Archive storage

Through Databases

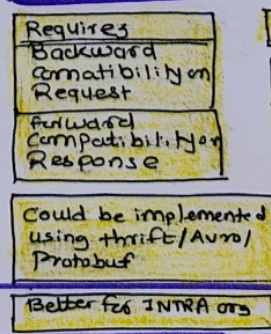


Through Services

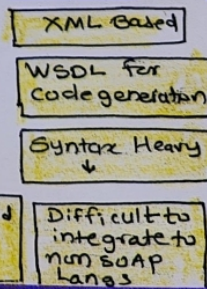
Background



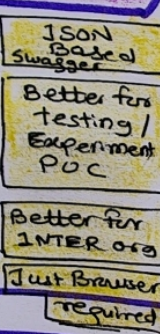
RPC protocol



SOAP protocol

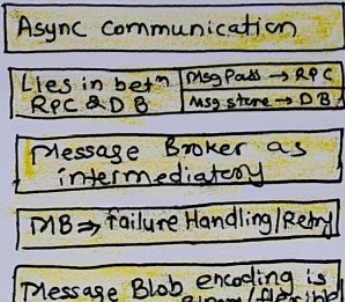


REST protocol

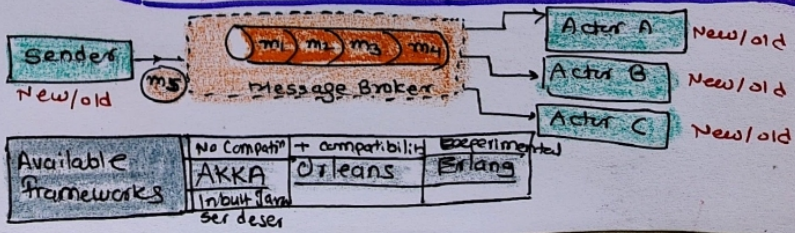


Through Message brokers

Background



Message Broker + Distributed Actor Framework



5. REPLICATION

(Part 1)

Summary

Why

Disconnected operations

High Availability

Reduce Latency

achieve Scalability

Algorithms

① Single Leader

② Multi Leader

③ Leaderless

Types

① Sync

② Async

③ Semi-Sync

Consistency

① Read after write

② Monotonic Reads

③ Consistent Prefix Read

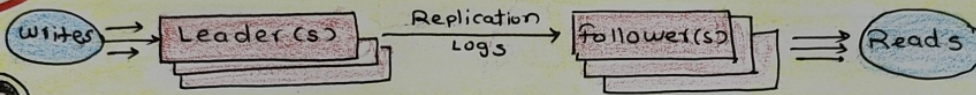
Log Replica

① SQL Statement

② Evaluated Statement
DB Ready from

③ Evaluated Statement
Logical form
Encode-Decode

④ Trigger or App code Based



CONCEPTS

Consistency

Leader & Follower data in **same** state.

Eventual Consistency

Relatively outdated replicas which provides high availability & low latency. Local writes at leader get sent **Async** as soon as possible.

Monotonic Reads

Same Read calls to multiple different **Async** follower replicas may potentially lead to different outputs.

Causal Dependency

Some data depends on other (cause) hence replication of 'cause' should happen before.

Sync/Async Replication

Sync $\Rightarrow$ Write Commit wait until **ALL** replication
Async $\Rightarrow$ **Immediate** write replication

Failover

Another follower **take over** as 'leader' in case of existing leader failure. Operator choose 'Manual' way of failover due to issues viz. split brain, delayed handoff, no right balanced timeout value for indicating Leader failure.

Split Brain

In single leader replication, **2 nodes** act as **Leader** leading to data loss and/or data corruption

Write Conflicts

Same key/datum updated on multiple leaders with **'DIFFERENT'** values.

Replication Topology

In multi-leader replication, communication path along with writes are forwarded across 'multiple' Leaders.

Quorum

Smallest number of $(W+R)$ replicas success. "Quorum condition"

Sloppy Quorum

In leaderless replication, Smallest number of (W, R) replicas succeeded with Non-DB nodes as temp measure in case of DB node down

Version Vector

Local $\rightarrow$ keep track of value changes through versions
Central $\rightarrow$ Central (Some special replica) keep track of global picture. & sends All version vectors to DB client.

ALGORITHMS

SINGLE LEADER

Single Leader Replication

Setup

Leader

Follower

Handle Failures

Replication Log impls

SQL Statement

DB raw log / Disk

DB Logical Log

App / Trigger based

Problems with Replication LAG

Quick Read data after write

Monotonic Reads

consistent Prefix Read

faulty solutions

What

write & Read same record instantly

Risk

Stale data Read

Solution

Choose LEADER for such Reads

What

Same READ requests to multiple followers

Risk

Async followers - Stale Reads

Solution

Intelligently CHOOSE SAME follower (single)

What

violation of causality old (cause) data is stale

Risk

Laggy follower return stale 'Cause'

Solution

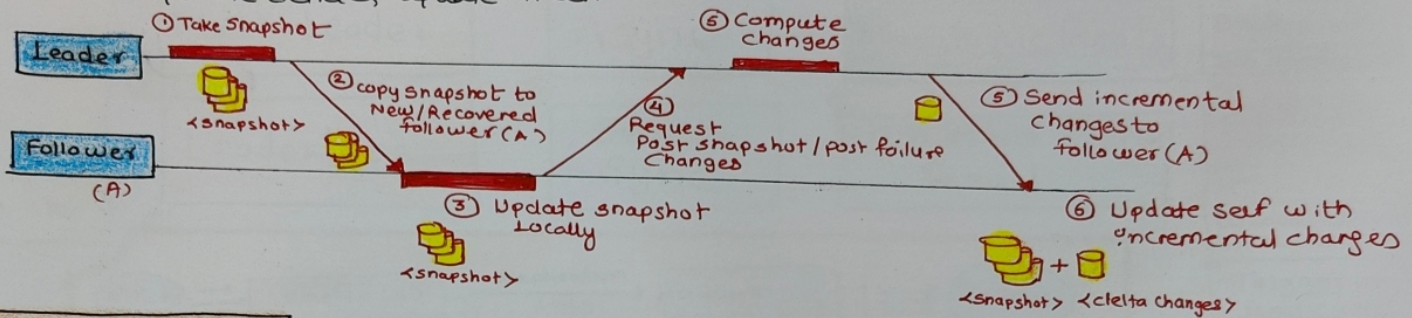
Track & write to follower

5. REPLICATION (part 2)

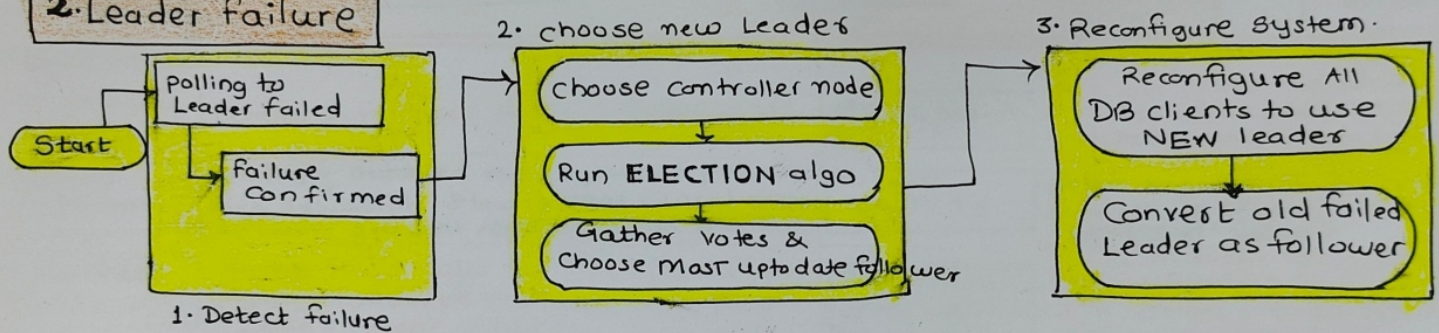
Single Leader Replication Failure Handling

1. Follower Failure

- Same as follower (New) setup, as below (diagram)
- Newly Setup follower will request delta changes, post <seq-id> before crash,
- It will accept the delta & update itself.



2. Leader failure



Algorithms - Multi Leader

Use cases

① Multi Data Center
Leader in each DC
log replication in Private Net

② Offline Clients
offline mobile calendar
sync with internet as available

③ Collaborative Editing
Multiple writers - same doc
Acquire locks on PARTS of doc

Merits

① High Performance - Less network latency

② High Availability - other DC leader as backup

③ Tolerance to N/w issues - log replication is safer in private than public net

Topologies

① CIRCULAR
1:1 replication → →
Demerit → Less fault tolerant

② STAR
Centralized, less invasive
Demerit → SPOC issue

③ ALL-TO-ALL
All:All - High fault tolerant
Demerit → Causality challenge

Biggest Challenge Handling write Conflicts Solutions

① Fully Synchronous way
Compromises High availability
Prefer Single Leader instead

② Avoid conflicts (Recommended)
Avoid specific record write to specific leader
Edge case → Leader change

③ Use <write-ID> per write
Give highest ID Preference
Let App Code resolve it
could lead to data loss due to clock failures

④ Custom Logic
Let App decide it the way we do for resolving multi-version dependencies

Avoid Multi leader Replication Unless you have Any of these use cases; since it introduces multiple risks, challenges & extra efforts.

5. REPLICATION

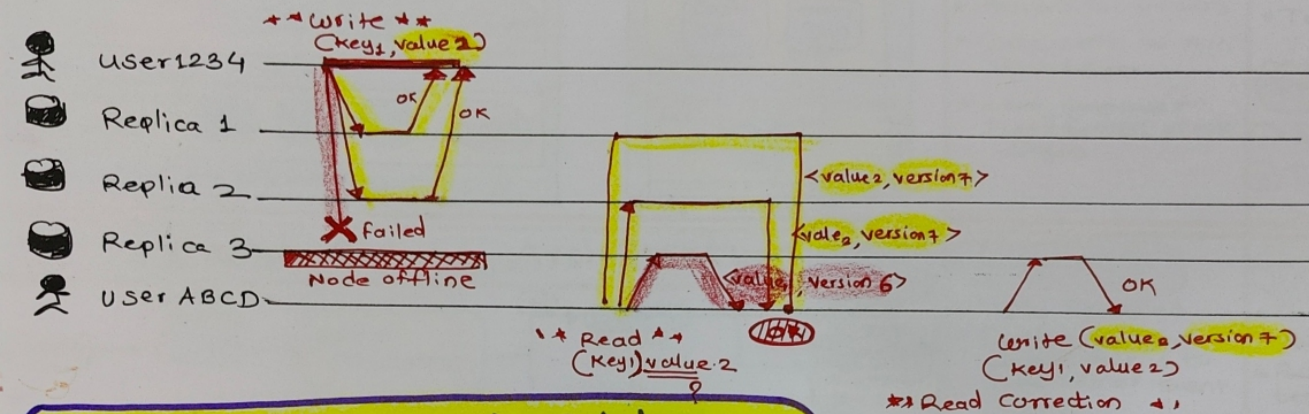
(Part - 3)

Algorithms - Leaderless Replication

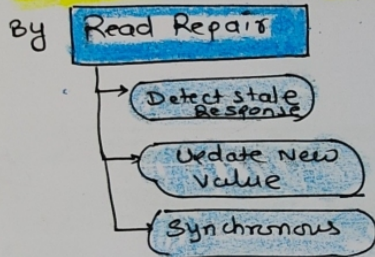
Flavors

(A) Client write to Replica

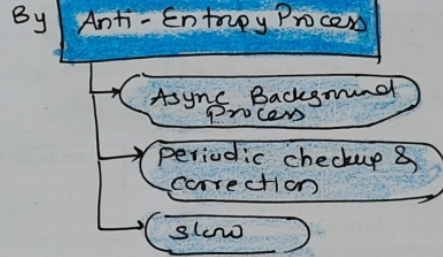
(B) write Using Coordinator Node



How offline/failed Node Catches Up

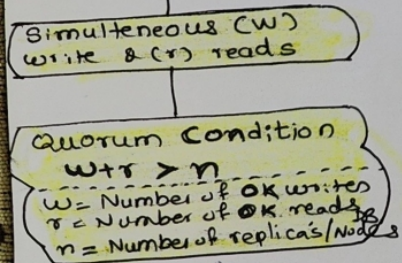


OR



Quorum Consistency

How does it work



Limitations

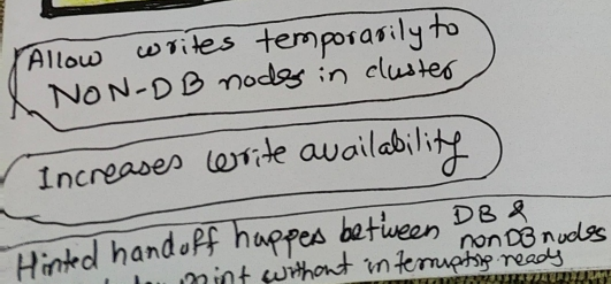
$w + r = \frac{n+1}{2}$ ensures at least 1 overlapping Read & write replica But.....

if, you choose these values manually then chances of Human error are high

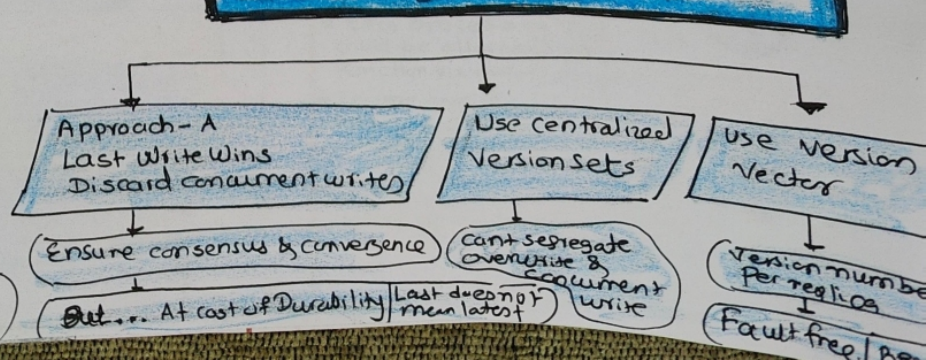
Leaderless Replication Use Cases

Multi Data Center Since slow / failed / Async partial writes are OK

'Sloppy' Quorums & Hinted Handoff

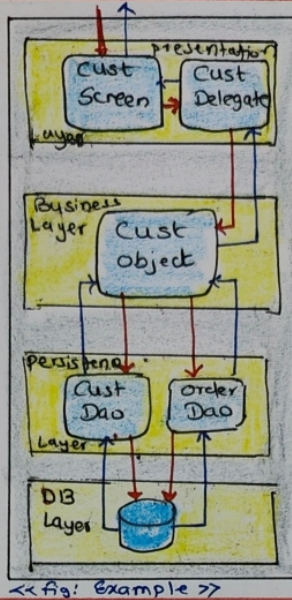
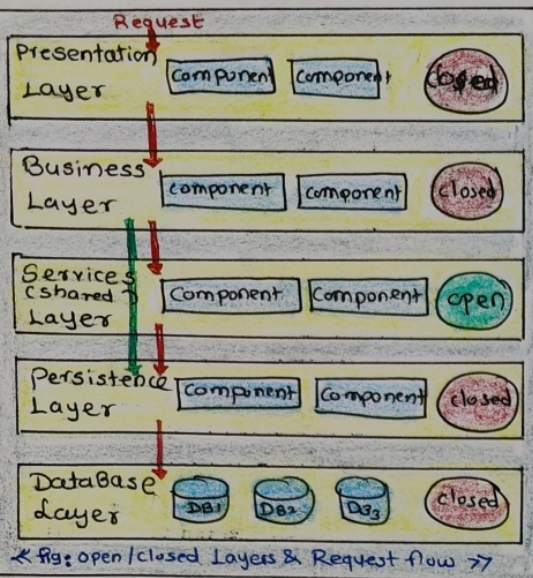


Detecting Concurrent Writes



Software Architecture Patterns ①

1) Layered Architecture



Summary

- ★ Separation of concern
- ★ open/closed layers
- ★ Loosely coupled —
- ★ Open layer for - shared services Passthrough
- ★ Layers varies as per app complexity.

Components

- ★ Horizontal Layers
- ★ Components within each Layer
- ★ <Req, Response> transformers across Layers

Usecase S

- ★ Best starting for most app
- ★ Can be fitted in microservices pattern
- ★ Small - mid scale app

Anti Patterns

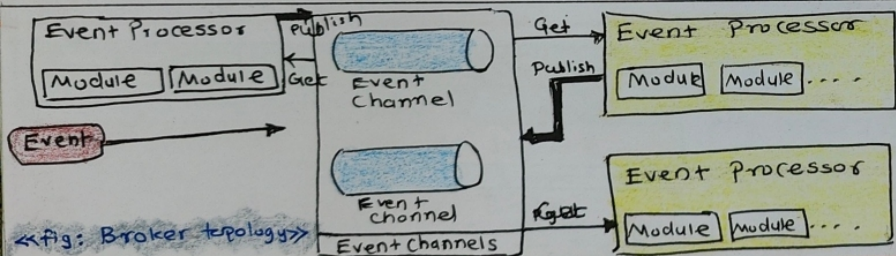
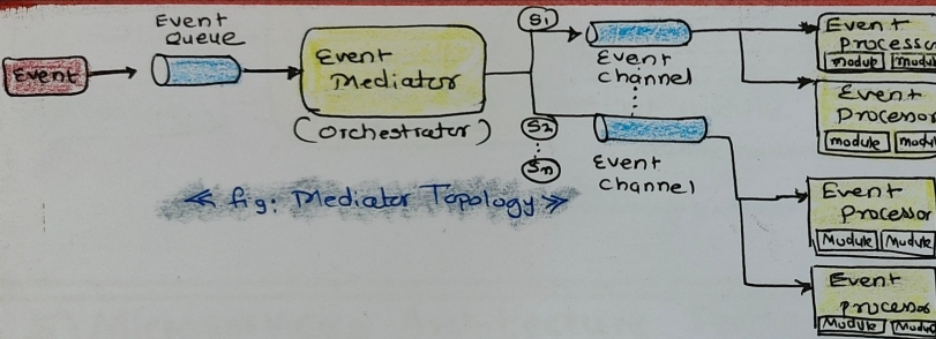
- ★ Passthrough Layers (If 20% traffic is Passthrough then use else if 80% is Passthrough → don't use)

- ★ Tendency to become monolithic

Best Suited for

- ① Developmentability
- ② Testability

2) Event Driven Architecture



Summary

- ★ Highly Decoupled
- ★ Common to have several dozens to hundreds of event Queues.

- ★ Initial event (vs) Processing Event

Mediator topology

- ★ Event Mediator as orchestrator for orchestrating steps from initial event (S1, S2, ..., Sn)
- ★ Known Implementations - Apache Camel, Mule ESB, Spring Integration

Broker Topology

- ★ No Central orchestrator
- ★ Used in case of relay/race chaining of events.

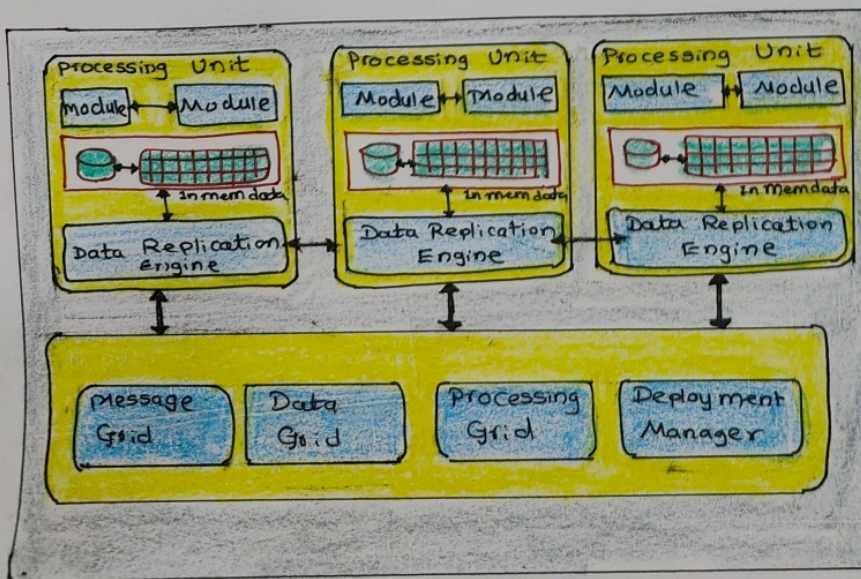
Use cases

- ★ Complex pattern to implement
- ★ Require specialized test data generator
- ★ Avoid - In case you need atomic transaction; or implement Rollbacks
- ★ Right for use case where its possible to divide task into parallel/sequential but independent processing units
- ★ Also in loosely coupled services integ.

Best Suited for

- ① Agility
- ② Deployment
- ③ Performance
- ④ Scalability

3) Space Based Architecture



Summary

- ★ Overcomes scalability Challenges.
- ★ Most Advanced
- ★ e.g. Amazon Honeycomb
- ★ Solves scalability & concurrency issues
- ★ NO Central DB hence near infinite Space.

Processing Unit

- ★ Contains frontend & backend business logic
- ★ Each processing unit could be allocated per function viz order, shipment
- ★ In memory Grid is <Replicated> Across multiple processing units
- ★ <optional> persistent store for failover

Virtualized Middleware

- ★ Handles housekeeping & communication
- ★ Message Grid → Determines active 'PU' & fwd request to it
- ★ Data Grid → Most important / interact with 'PU' internal Data Replication Engine
- ★ Processing Grid → <Optional> for interacting & orchestrating among PUs
- ★ Deployment Manager → Dynamic start/shutdown PUs as per load

Good for small web-based App with variable load.

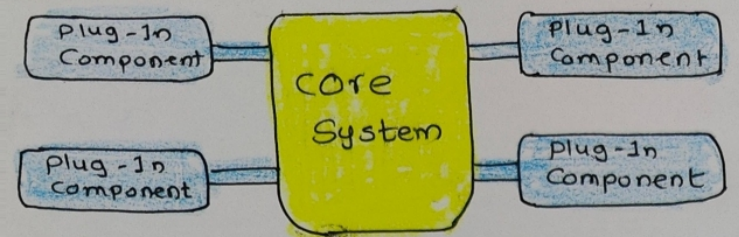
frameworks - Java Spaces, GemFire, GigaSpaces

Best Suited for

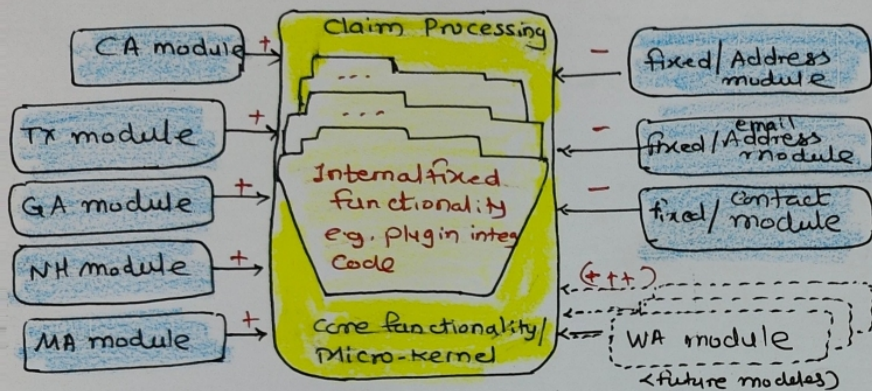
- ① Agility
- ② Deploy
- ③ performance
- ④ scalability

Software Architecture Patterns 2

4) Microkernel / plugin Based Pattern



«Fig: Microkernel - plugin based pattern»



«Fig: Example Microkernel processing pattern»

Summary

- * Preferred mainly in product based apps
- * Also applicable to shared development / Rule - evaluation / command type pattern use cases e.g. CASE plugin in Amazon ordering workflow
- * Major components
 - ① Core System / Microkernel
 - ② plugin components

* This pattern can be embedded inside any other pattern.

Best suited for

- 1> Agility
- 2> Deployment
- 3> Testing
- 4> performance

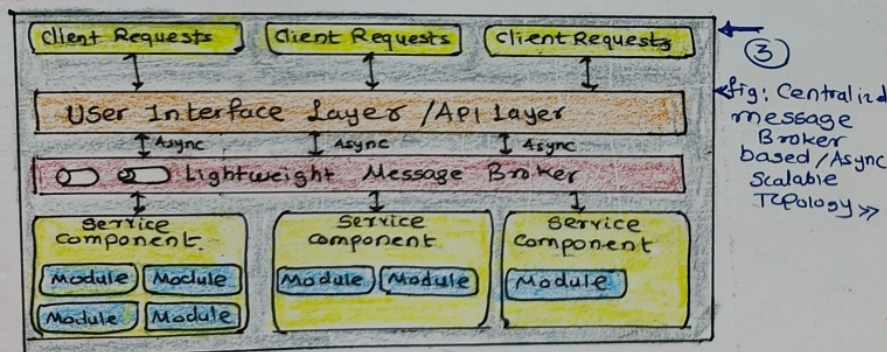
Challenges

- * Plugins are mainly "Additional" functionality on top of 'core' system functionality
- * Plugin modules can be connected using webservice, event binding dependency in 'pom', etc.
- * Not scalable because this is implemented for 'small' product (tightly coupled) which is non-distributed in nature.

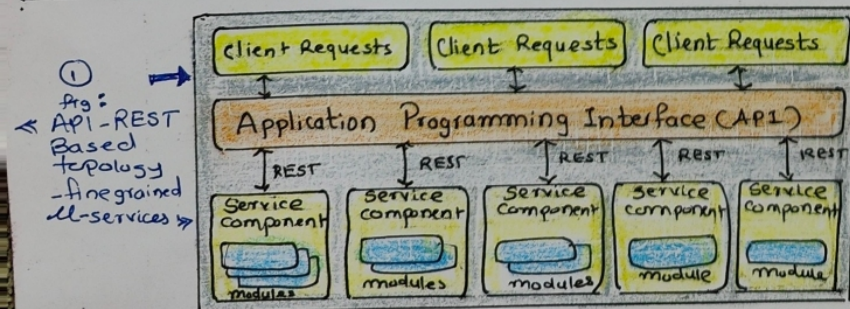
1> Contract governance between core & plugin modules

2> Not scalable because this is implemented for 'small' product (tightly coupled) which is non-distributed in nature.

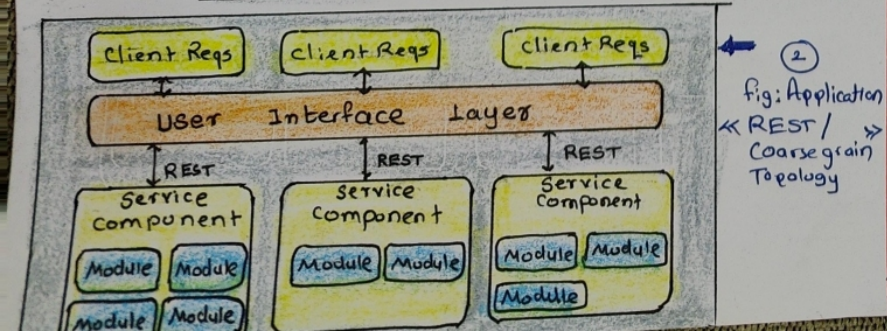
5) Microservices Architecture Pattern



③ «Fig: Centralized message Broker based / Async Scalable Topology»



① «Fig: API-REST Based topology - Finegrained services»



② «Fig: Application REST / Coarse grain Topology»

Summary

- * Separately deployable units / components (services)
- * Service component contains modules holding business logic
- * Distributed Architecture: Services interconnected thr' REST / RPC / etc.
- * Could be evolved from ① Monolithic (Single Service) or ② SOA (Coarse grain services)
- * Better manageable than monolithic & SOA patterns
- * Determining right level of granularity
- * Coarsegrained => Requires inter-service calls & dependencies (Problem - Anti pattern)
- * Too Finegrained => Orchestration needs more complexity
- * More failure points

Merits

- * Real time deployments
- * Modularity
- * Distributed Architecture
- * Can embed both event-driven & levelled architectures within this pattern

Topologies

- ① API REST based
 - Useful for small self contained services thr' APIs.
 - Suitable for mid-high complex
- ② Application REST based
 - Services are coarse grained
 - Suitable for small - mid complex
- ③ Centralized messaging
 - Centralized message broker for large, complex app like youtube.

Best Suited for

- ① Agility
- ② Deployment
- ③ Testability
- ④ Scalability
- ⑤ Development